PdfFactory3 修正履歴：2005/10/21 NEC Soft, Ltd. 「FormCraft Pro. for Web」から「Form Manager V6.1」へのバージョンアップ に

表紙

項目	内容
文書名	基本設計書
システム名	
機能ID	COM-056
画面名	PdfFactory3 修正履歴：2005/10/21 NEC Soft, Ltd. 「FormCraft Pro. for Web」から「Form Manager V6.1」へのバージョンアップ に
会社名	
作成日	

概要

概要説明

PdfFactory3 修正履歴：2005/10/21 NEC Soft, Ltd. 「FormCraft Pro. for Web」から「Form Manager V6.1」へのバージョンアップ にの入力・表示・要求処理を行う。

情報	内容
対象種別	common
開始契機	呼出元からの要求
処理対象	入力値・画面状態

項目一覧

No	グループ	項目名	種類	入出力	DB連携項目	候補取得元	必須	形式・桁	説明

項目定義数：0項目。表示順と反復行の定義数を区別する。

イベント仕様

makeVector

処理概要

PdfFactory3 修正履歴：2005/10/21 NEC Soft, Ltd. 「FormCraft Pro. for Web」から「Form Manager V6.1」へのバージョンアップ にで「makeVector」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理8：詳細仕様参照）。

2. 関連情報を取得・設定する（処理9：詳細仕様参照）。

3. 繰返し条件「条件10を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

3.1. 繰返しの各回：関連情報を取得・設定する（処理11：詳細仕様参照）。

3.2. 繰返しの各回：繰返し条件「int i=0; i < 「対象項目」; i++」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

3.2.1. 繰返しの各回：「rs.getString(i+1) が等しい 未設定」に該当するか確認する。成立時と不成立時の手順を分けて行う。

3.2.1.1. 条件成立時：関連情報を取得・設定する（処理12：詳細仕様参照）。

3.2.1-別.1. 条件不成立時：関連情報を取得・設定する（処理13：詳細仕様参照）。

3.3. 繰返しの各回：関連情報を取得・設定する（処理14：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

makeVector

処理概要

PdfFactory3 修正履歴：2005/10/21 NEC Soft, Ltd. 「FormCraft Pro. for Web」から「Form Manager V6.1」へのバージョンアップ にで「makeVector」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理11：詳細仕様参照）。
2. 関連情報を取得・設定する（処理12：詳細仕様参照）。
3. 関連情報を取得・設定する（処理13：詳細仕様参照）。
4. 繰返し条件「条件14を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

4.1. 繰返しの各回：関連情報を取得・設定する（処理15：詳細仕様参照）。

4.2. 繰返しの各回：関連情報を取得・設定する（処理16：詳細仕様参照）。

4.3. 繰返しの各回：繰返し条件「int i=0; i < 「対象項目」; i++」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

4.3.1. 繰返しの各回：「rs.getString(i+1) が等しい 未設定」に該当するか確認する。成立時と不成立時の手順を分けて行う。

4.3.1.1. 条件成立時：関連情報を取得・設定する（処理17：詳細仕様参照）。

4.3.1-別.1. 条件不成立時：関連情報を取得・設定する（処理18：詳細仕様参照）。

4.4. 繰返しの各回：関連情報を取得・設定する（処理19：詳細仕様参照）。

1. 「条件20を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
条件21を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

到達条件	結果
条件22を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す

makeFieldData

処理概要

PdfFactory3 修正履歴：2005/10/21 NEC Soft, Ltd. 「FormCraft Pro. for Web」から「Form Manager V6.1」へのバージョンアップ にで「makeFieldData」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理26：詳細仕様参照）。
2. 関連情報を取得・設定する（処理27：詳細仕様参照）。
3. 関連情報を取得・設定する（処理28：詳細仕様参照）。
4. 関連情報を取得・設定する（処理29：詳細仕様参照）。
5. 関連情報を取得・設定する（処理30：詳細仕様参照）。
6. 関連情報を取得・設定する（処理31：詳細仕様参照）。
7. 「対象項目がない、または条件32を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

7-別.1. 条件不成立時：関連情報を取得・設定する（処理33：詳細仕様参照）。

7-別.2. 条件不成立時：繰返し条件「条件34を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

7-別.2.1. 繰返しの各回：関連情報を取得・設定する（処理35：詳細仕様参照）。

1. 関連情報を取得・設定する（処理36：詳細仕様参照）。
2. 関連情報を取得・設定する（処理37：詳細仕様参照）。

3. 関連情報を取得・設定する（処理38：詳細仕様参照）。

4. 繰返し条件「条件39を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

11.1. 繰返しの各回：関連情報を取得・設定する（処理40：詳細仕様参照）。

11.2. 繰返しの各回：「条件41を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

11.2.1. 条件成立時：関連情報を取得・設定する（処理42：詳細仕様参照）。

11.3. 繰返しの各回：関連情報を取得・設定する（処理43：詳細仕様参照）。

11.4. 繰返しの各回：関連情報を取得・設定する（処理44：詳細仕様参照）。

11.5. 繰返しの各回：繰返し条件「条件45を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

11.5.1. 繰返しの各回：関連情報を取得・設定する（処理46：詳細仕様参照）。

11.5.2. 繰返しの各回：「条件47を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

11.5.2.1. 条件成立時：関連情報を取得・設定する（処理48：詳細仕様参照）。

11.5.3. 繰返しの各回：関連情報を取得・設定する（処理49：詳細仕様参照）。

11.5.4. 繰返しの各回：関連情報を取得・設定する（処理50：詳細仕様参照）。

1. 指定した条件でデータを照会する。

2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

writePdf

処理概要

PdfFactory3 修正履歴：2005/10/21 NEC Soft, Ltd. 「FormCraft Pro. for Web」から「Form Manager V6.1」へのバージョンアップ にで「writePdf」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目」を設定する。
2. 関連情報を取得・設定する（処理39：詳細仕様参照）。
3. 関連情報を取得・設定する（処理40：詳細仕様参照）。
4. 関連情報を取得・設定する（処理41：詳細仕様参照）。
5. 関連情報を取得・設定する（処理42：詳細仕様参照）。
6. 関連情報を取得・設定する（処理43：詳細仕様参照）。
7. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

7.1. 通常処理：関連情報を取得・設定する（処理44：詳細仕様参照）。

7.2. 通常処理：「対象項目」を設定する。

7.3. 通常処理：関連情報を取得・設定する（処理45：詳細仕様参照）。

7.4. 通常処理：関連情報を取得・設定する（処理46：詳細仕様参照）。

7.5. 通常処理：関連情報を取得・設定する（処理47：詳細仕様参照）。

7.6. 通常処理：関連情報を取得・設定する（処理48：詳細仕様参照）。

7.7. 通常処理：関連情報を取得・設定する（処理49：詳細仕様参照）。

7.8. 通常処理：関連情報を取得・設定する（処理50：詳細仕様参照）。

7.9. 通常処理：関連情報を取得・設定する（処理51：詳細仕様参照）。

7.10. 通常処理：関連情報を取得・設定する（処理52：詳細仕様参照）。

7.11. 通常処理：関連情報を取得・設定する（処理53：詳細仕様参照）。

7.12. 通常処理：通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

7.12.1. 通常処理：関連情報を取得・設定する（処理54：詳細仕様参照）。

7.12-例外1. 例外条件「FMPropertyException e」に該当した場合に次の手順を行う。型名は原本の表記。

7.12-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

7.12-例外2. 例外条件「FMPrintException e」に該当した場合に次の手順を行う。型名は原本の表記。

7.12-例外2.1. 例外発生時：関連情報を取得・設定する（処理55：詳細仕様参照）。

7.12-例外2.2. 例外発生時：関連情報を取得・設定する（処理56：詳細仕様参照）。

7.12-例外2.3. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

7.13. 通常処理：関連情報を取得・設定する（処理57：詳細仕様参照）。

7.14. 通常処理：繰返し条件「条件58を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

7.14.1. 繰返しの各回：「status が等しい 0、またはstatus が等しい 1000008、またはstatus が等しい 1000019」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

7.14.1.1. 条件成立時：関連情報を取得・設定する（処理59：詳細仕様参照）。

7.14.1.2. 条件成立時：関連情報を取得・設定する（処理60：詳細仕様参照）。

7.14.2. 繰返しの各回：関連情報を取得・設定する（処理61：詳細仕様参照）。

7.14.3. 繰返しの各回：関連情報を取得・設定する（処理62：詳細仕様参照）。

7.15. 通常処理：関連情報を取得・設定する（処理63：詳細仕様参照）。

7.16. 通常処理：「status が等しい 0」に該当するか確認する。成立時と不成立時の手順を分けて行う。

7.16.1. 条件成立時：「対象項目がある」に該当するか確認する。成立時と不成立時の手順を分けて行う。

7.16.1.1. 条件成立時：「((Integer)response.get("fm-status")).intValue() が等しい 0」に該当するか確認する。成立時と不成立時の手順を分けて行う。

7.16.1.1.1. 条件成立時：関連情報を取得・設定する（処理64：詳細仕様参照）。

7.16.1.1.2. 条件成立時：関連情報を取得・設定する（処理65：詳細仕様参照）。

7.16.1.1-別.1. 条件不成立時：関連情報を取得・設定する（処理66：詳細仕様参照）。

7.16.1-別.1. 条件不成立時：「利用できるコンポーサがありません。」を設定する。

7.16-別.1. 条件不成立時：「ジョブが中断された。」を設定する。

7.17. 通常処理：「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

7.17.1. 条件成立時：値に応じた分岐の詳細は原本対応を参照する。

7.18. 通常処理：「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

7.18.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

7-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

7-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理67：詳細仕様参照）。
2. 関連情報を取得・設定する（処理68：詳細仕様参照）。
3. 関連情報を取得・設定する（処理69：詳細仕様参照）。
4. 関連情報を取得・設定する（処理70：詳細仕様参照）。

5. 繰返し条件「(nread = stream.read(buf)) が異なる -1」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

12.1. 繰返しの各回：関連情報を取得・設定する（処理71：詳細仕様参照）。

12.2. 繰返しの各回：関連情報を取得・設定する（処理72：詳細仕様参照）。

1. 関連情報を取得・設定する（処理73：詳細仕様参照）。
2. 関連情報を取得・設定する（処理74：詳細仕様参照）。
3. 関連情報を取得・設定する（処理75：詳細仕様参照）。

結果・例外時の扱い

到達条件	結果
条件76を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
条件77を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
対象項目がある	処理の異常を通知し、この経路を終了する
条件78を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

補足説明

適用条件

各仮定のQA番号を確認する。静的に確認できた操作・条件を適用する。

処理・条件の原本対応

文書内の条件番号・処理番号は次の原本表記に対応する。完全な業務用語への置換を行っていない箇所はQA・reportで確認する。

イベント	参照番号	原本条件・処理
makeVector	条件1	rs.next()
makeVector	処理2	dataSource.addElement(row)
makeVector	処理3	row.addElement("")
makeVector	処理4	row.addElement(rs.getString(i+1))
makeVector	処理5	Vector row = new Vector()
makeVector	処理6	ResultSetMetaData meta = rs.getMetaData()
makeVector	処理7	Vector dataSource = new Vector()
makeVector	処理8	Vector dataSource = new Vector()
makeVector	処理9	ResultSetMetaData meta = rs.getMetaData()
makeVector	条件10	rs.next()
makeVector	処理11	Vector row = new Vector()
makeVector	処理12	row.addElement("")
makeVector	処理13	row.addElement(rs.getString(i+1))
makeVector	処理14	dataSource.addElement(row)
makeVector	条件1	selectDataUmu.equals("NASHI")
makeVector	条件2	rs.next()
makeVector	処理3	dataSource.addElement(row)

イベント	参照番号	原本条件・処理
makeVector	処理4	row.addElement("")
makeVector	処理5	row.addElement(rs.getString(i+1))
makeVector	処理6	Vector row = new Vector()
makeVector	処理7	selectDataUmu = "ARI"
makeVector	処理8	String selectDataUmu = "NASHI"
makeVector	処理9	ResultSetMetaData meta = rs.getMetaData()
makeVector	処理10	Vector dataSource = new Vector()
makeVector	処理11	Vector dataSource = new Vector()
makeVector	処理12	ResultSetMetaData meta = rs.getMetaData()
makeVector	処理13	String selectDataUmu = "NASHI"
makeVector	条件14	rs.next()
makeVector	処理15	selectDataUmu = "ARI"
makeVector	処理16	Vector row = new Vector()
makeVector	処理17	row.addElement("")
makeVector	処理18	row.addElement(rs.getString(i+1))
makeVector	処理19	dataSource.addElement(row)
makeVector	条件20	selectDataUmu.equals("NASHI")
makeVector	条件21	selectDataUmu.equals("NASHI")
makeVector	条件22	NOT (selectDataUmu.equals("NASHI"))

イベント	参照番号	原本条件・処理
makeFieldData	条件1	int row = 0; row < variableData.size(); row++
makeFieldData	条件2	int col = 0; col < rowdata.size(); col++
makeFieldData	処理3	isFirstCol = false
makeFieldData	処理4	sb.append(QT + dt + QT)
makeFieldData	処理5	sb.append(SP)
makeFieldData	条件6	!isFirstCol
makeFieldData	処理7	String dt = rowdata.get(col).toString().replaceAll("\\","\\\\")
makeFieldData	処理8	List rowdata = (List)variableData.get(row)
makeFieldData	処理9	isFirstRow = false
makeFieldData	処理10	sb.append("\r\n")
makeFieldData	条件11	!isFirstRow
makeFieldData	処理12	isFirstCol = true
makeFieldData	処理13	boolean isFirstCol = true
makeFieldData	処理14	boolean isFirstRow = true
makeFieldData	処理15	sb.append("[Body Data Section]\r\n")
makeFieldData	条件16	int i=0 ; i < fixedData.length ; i++
makeFieldData	処理17	sb.append(fixedData[i][0] + "=" + QT + fixedData[i][1] + QT + "\r\n")
makeFieldData	処理18	sb.append("[Fixed Data Section]\r\n")
makeFieldData	条件19	fixedData[0][0].equals("")

イベント	参照番号	原本条件・処理
makeFieldData	処理20	sb.append("VERSION=5.0\r\n")
makeFieldData	処理21	sb.append("[Control Section]\r\n")
makeFieldData	処理22	StringBuffer sb = new StringBuffer()
makeFieldData	処理23	byte[] fielddata = null
makeFieldData	処理24	final String QT = """
makeFieldData	処理25	final String SP = ","
makeFieldData	処理26	final String SP = ","
makeFieldData	処理27	final String QT = """
makeFieldData	処理28	byte[] fielddata = null
makeFieldData	処理29	StringBuffer sb = new StringBuffer()
makeFieldData	処理30	sb.append("[Control Section]\r\n")
makeFieldData	処理31	sb.append("VERSION=5.0\r\n")
makeFieldData	条件32	fixedData[0][0].equals("")
makeFieldData	処理33	sb.append("[Fixed Data Section]\r\n")
makeFieldData	条件34	int i=0 ; i < fixedData.length ; i++
makeFieldData	処理35	sb.append(fixedData[i][0] + "=" + QT + fixedData[i][1] + QT + "\r\n")
makeFieldData	処理36	sb.append("[Body Data Section]\r\n")
makeFieldData	処理37	boolean isFirstRow = true
makeFieldData	処理38	boolean isFirstCol = true

イベント	参照番号	原本条件・処理
makeFieldData	条件39	int row = 0; row < variableData.size(); row++
makeFieldData	処理40	isFirstCol = true
makeFieldData	条件41	!isFirstRow
makeFieldData	処理42	sb.append("\r\n")
makeFieldData	処理43	isFirstRow = false
makeFieldData	処理44	List rowdata = (List)variableData.get(row)
makeFieldData	条件45	int col = 0; col < rowdata.size(); col++
makeFieldData	処理46	String dt = rowdata.get(col).toString().replaceAll("\\","\\\\")
makeFieldData	条件47	!isFirstCol
makeFieldData	処理48	sb.append(SP)
makeFieldData	処理49	sb.append(QT + dt + QT)
makeFieldData	処理50	isFirstCol = false
writePdf	処理1	agent.removeJob(job)
writePdf	処理2	job.clear()
writePdf	処理3	out.flush()
writePdf	処理4	out.write(buf, 0, nread)
writePdf	処理5	len =len + nread
writePdf	処理6	int len = 0
writePdf	処理7	int nread=0

イベント	参照番号	原本条件・処理
writePdf	処理8	byte[] buf = new byte[2048]
writePdf	処理9	out = res.getOutputStream()
writePdf	例外	job.getStatus()
writePdf	処理11	stream = operator.getInputStream()
writePdf	処理12	FMResponseOperator operator = job.getResponseOperator()
writePdf	処理13	errorMessage = (String)response.get("fm-message")
writePdf	処理14	String errorMessage = null
writePdf	条件15	int i = 0; i < 20; i++
writePdf	処理16	status = job.getStatus()
writePdf	処理17	Thread.sleep(1000)
writePdf	処理18	break
writePdf	処理19	response = job.getResponse()
writePdf	処理20	int status = job.getStatus()
writePdf	処理21	int runtimeCode = e.getRuntimeErrorCode()
writePdf	処理22	String errorMessage = e.getMessage()
writePdf	処理23	job.asynOverlay()
writePdf	処理24	FMResponse response = null
writePdf	処理25	request.put("fm-outputtype", "pdf")
writePdf	処理26	request.put("fm-action", "view")

イベント	参照番号	原本条件・処理
writePdf	処理27	request.put("fm-target", "client")
writePdf	処理28	request.put("fm-fielddata", testdata)
writePdf	処理29	request.put("fm-formfilename", formatFileName)
writePdf	処理30	request = job.getRequest()
writePdf	処理31	job = agent.newJob()
writePdf	処理32	agent = new FMComposerAgent(null, null, null)
writePdf	処理33	byte[] testdata = makeFieldData(fixedData, variableData)
writePdf	処理34	ServletOutputStream out = null
writePdf	処理35	InputStream stream = null
writePdf	処理36	FMRequest request = null
writePdf	処理37	FMJob job = null
writePdf	処理38	FMComposerAgent agent = null
writePdf	処理39	FMComposerAgent agent = null
writePdf	処理40	FMJob job = null
writePdf	処理41	FMRequest request = null
writePdf	処理42	InputStream stream = null
writePdf	処理43	ServletOutputStream out = null
writePdf	処理44	byte[] testdata = makeFieldData(fixedData, variableData)
writePdf	処理45	agent = new FMComposerAgent(null, null, null)

イベント	参照番号	原本条件・処理
writePdf	処理46	job = agent.newJob()
writePdf	処理47	request = job.getRequest()
writePdf	処理48	request.put("fm-formfilename", formatFileName)
writePdf	処理49	request.put("fm-fielddata", testdata)
writePdf	処理50	request.put("fm-target", "client")
writePdf	処理51	request.put("fm-action", "view")
writePdf	処理52	request.put("fm-outputtype", "pdf")
writePdf	処理53	FMResponse response = null
writePdf	処理54	job.asynOverlay()
writePdf	処理55	String errorMessage = e.getMessage()
writePdf	処理56	int runtimeCode = e.getRuntimeErrorCode()
writePdf	処理57	int status = job.getStatus()
writePdf	条件58	int i = 0; i < 20; i++
writePdf	処理59	response = job.getResponse()
writePdf	処理60	break
writePdf	処理61	Thread.sleep(1000)
writePdf	処理62	status = job.getStatus()
writePdf	処理63	String errorMessage = null
writePdf	処理64	FMResponseOperator operator = job.getResponseOperator()

イベント	参照番号	原本条件・処理
writePdf	処理65	stream = operator.getInputStream()
writePdf	処理66	errorMessage = (String)response.get("fm-message")
writePdf	処理67	out = res.getOutputStream()
writePdf	処理68	byte[] buf = new byte[2048]
writePdf	処理69	int nread=0
writePdf	処理70	int len = 0
writePdf	処理71	len =len + nread
writePdf	処理72	out.write(buf, 0, nread)
writePdf	処理73	out.flush()
writePdf	処理74	job.clear()
writePdf	処理75	agent.removeJob(job)
writePdf	条件76	FMPropertyException e
writePdf	条件77	FMPrintException e
writePdf	条件78	Exception e